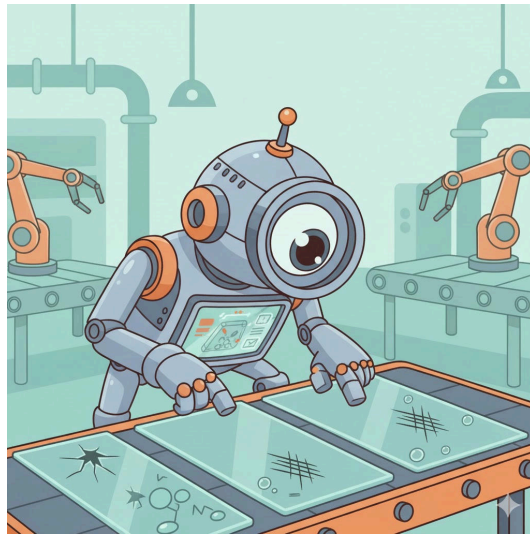


Glass Defect Detection: Evaluating Object Detection Models



Confidential

Projectwork 2

by

Florian Vollmer

Bachelors 2023

Submission date: 2025-12-11

Study Direction: Applied Digital Life Science

Supervisors:

Dr. Stefan Glüge

ZHAW Life Sciences und Facility Management, Wädenswil

Imprint

Recommended Citation:

Florian Vollmer (2025). *Glass Defect Detection: Evaluating Object Detection Models*

.

Keywords: Deep Learning, Object Detectors, Image Processing, Evaluation Metrics

Abstract

Write after work is written

Table of Contents

List of abbreviations	4
1. Introduction	5
1.1. Context and Motivation	5
1.2. Overall Concept	6
1.3. Research Questions	6
1.4. State of the Art	7
2. Methodology	10
2.1. Datasets and Assumptions	10
2.2. Model Selection	12
2.3. Model Mechanics	13
2.4. Model Training	14
2.5. Model Testing	15
2.6. Evaluation Metrics	15
3. Results	17
3.1. Model Results and Statistical Evaluation	17
4. Discussion	18
4.1. Interpretation of Results	18
4.2. Research Questions & Assumptions	18
5. Conclusion	19
5.1. Summary of Key Findings	19
5.2. Limitations and Challenges	19
5.3. Future Research Directions	19
6. References	21
7. Lists	22
List of Figures	22
List of Tables	22
8. Attachments	23

8.1. Raw Data	23
---------------------	----

List of abbreviations

Potential abbreviations

1. Introduction

1.1. Context and Motivation

In the current technological landscape, the implementation of modern technologies is indispensable for ensuring precise and automated processes. Within the industrial sector, quality control represents a critical success factor. Specifically in pharmaceuticals and medical technology, the integrity of the primary packaging material (typically glass containers) is crucial for product safety.

The primary material used for medical and chemical glass containers generally consists of borosilicate glass. However, this material can exhibit or contain production-related defects, including particulate inclusions, air bubbles, or flexible fragments known as Lamellae (Foglia, Prete, and Zanda 2015).

The relevance of these quality deficiencies is highlighted by the consequences demonstrated by (Foglia, Prete, and Zanda 2015): *“In recent years, more than 20 product recalls have been associated with glass issues, and over 100 million units of drugs packaged in vials or syringes have been withdrawn from the market.”* This underscores the urgent necessity of reliable detection methods for such flaws.

The rapid advancement of Artificial Intelligence (AI) and fundamental mechanisms like Deep Learning allows for the application of so-called object detectors for the automated recognition of the aforementioned defects. The attractiveness of these AI-based image recognition methods is continuously increasing (Bhatt et al. 2021). However, as these procedures still represent a nascent field of research, comprehensive validation studies on their suitability and the achievability of the necessary performance in industrial use are lacking.

A central challenge in training robust models is the scarcity of extensive datasets (Bhatt et al. 2021). Specifically, the naturally limited availability of data from defective products, caused by the significant imbalance in favor of intact units, complicates the creation of an adequate training dataset (Bhatt et al. 2021).

This paper serves as an exploratory contribution to the evaluation of the current feasibility of AI-supported defect detection. The objective of this work is the comparison of available models on the market and the existence of suitable datasets, in order to provide a well-founded statement on the realisability of precise defect determination.

1.2. Overall Concept

The overall objective of this work was to conduct a quantitative performance comparison of two different object detection models under varying data conditions. The experimental procedure was based on the following fundamental steps:

1. **Model selection and preparation:** Two specific, pre-trained foundation models from the class of object detectors were selected and initialized for evaluation.
2. **Data management and fine-tuning:** To test the models' generalizability, they were each adapted to the specific classification task (defect detection) using two discrete datasets (dataset A and dataset B) via a fine-tuning process.
3. **Performance evaluation:** The performance of the four trained model-dataset combinations was then evaluated on a separate, previously unseen test dataset. The performance was determined based on established quantitative metrics for object detection.
4. **Comparative analysis:** Finally, the generated results were visualized and subjected to a detailed comparative analysis. The aim was to identify the robustness, strengths, and weaknesses of the model architectures examined in relation to glass defect detection.

1.3. Research Questions

Based on the challenges mentioned in **Context and Motivation**, this study tries to answer following research questions:

- How does the performance differ between the two investigated object detection models (Model X vs. Model Y), measured by Mean Average Precision (mAP), in detecting defects in the different datasets?
- Which performance benchmarks (e.g., minimum precision and recall values) must be achieved to deem the resulting reliability of the models as sufficient for deployment in real-world quality control scenarios?

1.4. State of the Art

The quote from (Buhl (2024)) explains quite well what Object detection is: “*Object detection is a critical capability of computer vision that identifies and locates objects within an image or video. Unlike image classification, object detection not only classifies the objects in an image, but also identifies their location within the image by drawing a bounding box around each object.*”

Given the lack of specific literature concerning the performance of contemporary object detectors on glass vial defect inspection, this study must rely on fundamental model performance benchmarks established by previous research. These benchmarks are typically derived from models trained on broad, general-purpose datasets.

The following analysis focuses on the existing comparative statistics for the chosen architectures: YOLO and RF-DETR.

Model	Size	# Params.	GFLOPS	Latency (ms)	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
Real-Time Object Detection w/ NMS										
YOLOv8 (Jocher et al., 2023)	N	3.2M	8.7	2.1	35.2	49.2	38.3	15.8	38.8	51.3
YOLOv11 (Jocher et al., 2024)	N	2.6M	6.5	2.2	37.1	51.6	40.4	17.3	40.7	55.6
YOLOv8 (Jocher et al., 2023)	S	11.2M	28.6	2.9	42.4	57.6	46.0	22.2	47.1	59.6
YOLOv11 (Jocher et al., 2024)	S	9.4M	21.5	3.2	44.1	59.3	47.9	26.1	48.5	62.6
YOLOv8 (Jocher et al., 2023)	M	25.9M	78.9	5.4	47.3	62.5	51.5	27.5	52.9	65.1
YOLOv11 (Jocher et al., 2024)	M	20.1M	68.0	5.1	48.3	63.6	52.5	29.1	53.8	66.3
Open-Vocabulary Object Detection (Fully-Supervised Fine-Tuning)										
GroundingDINO (Liu et al., 2023)	T	173.0M	1008.3	427.6*	58.2	-	-	-	-	-
End-to-End Real-Time Object Detection										
LW-DETR (Chen et al., 2024a)	T	12.1M	21.4	1.9	42.9	60.7	45.9	22.7	47.3	60.0
D-FINE (Peng et al., 2024)	N	3.8M	7.3	2.1	42.7	60.2	45.4	22.9	46.6	62.1
RF-DETR (Ours)	N	30.5M	31.9	2.3	48.0	67.0	51.4	25.2	53.5	70.0
LW-DETR (Chen et al., 2024a)	S	14.6M	31.8	2.6	48.0	66.8	51.6	26.7	52.5	65.6
D-FINE (Peng et al., 2024)	S	10.2M	25.2	3.5	50.6	67.6	55.0	32.6	54.6	66.6
RF-DETR (Ours)	S	32.1M	59.8	3.5	52.9	71.9	57.0	32.0	58.3	73.0
RT-DETR (Zhao et al., 2024)	R18	36.0M	100.0	4.4	49.0	66.6	53.3	32.8	52.1	65.0
LW-DETR (Chen et al., 2024a)	M	28.2M	83.9	4.4	52.6	72.0	56.6	32.5	57.6	70.5
D-FINE (Peng et al., 2024)	M	19.2M	56.6	5.4	55.0	72.6	59.7	37.6	59.4	71.7
RF-DETR (Ours)	M	33.7M	78.8	4.4	54.7	73.5	59.2	36.1	59.7	73.8
RF-DETR (Ours)	2XL	126.9M	438.4	17.2	60.1	78.5	65.5	43.2	64.9	76.2

Figure 1: Comparison Table of different Object Detectors

The initial comparison in (Figure 1) provides a direct performance evaluation of various object detectors, including the two architectures selected for this paper (YOLOv11 Nano and RF-DETR Nano). Crucially, Non-Maximum Suppression (NMS), a fundamental post-processing algorithm, is utilised by YOLO to filter out overlapping, redundant bounding boxes, thereby ensuring that each object instance is identified only once (Ultralytics (2025)). The results published by Robinson et al. (2025) reveal significant architectural and performance differences between the nano variants: - Architectural Size: The RF-DETR

model possesses 30.5 million parameters (M), whereas the YOLO variant contains only 2.6M parameters. This disparity reflects a substantial difference in model complexity and memory footprint. - Detection Accuracy: Correspondingly, the RF-DETR model achieves a higher Average Precision (AP) of 48.0%, while the YOLO model reaches an AP of 37.1%. This trend is maintained across varying Intersection over Union (IoU) overlap requirements. These results directly reflect the discrepancy in file size: The RF-DETR Nano model is approximately 349MB, while the YOLOv11 Nano model is only 57MB. This size differential represents a significant factor in the decision-making process regarding model selection for resource-constrained real-world deployment.

Contrasting Benchmarks Interestingly, the benchmark table presented by Sapkota et al. (2025), shown in (Figure 2)

Models	Single-Class			Multi-Class		
	Precision	Recall	F1 Score	Precision	Recall	F1 Score
RF-DETR	0.8663	0.8828	0.8744	0.7652	0.8109	0.7874
YOLOv12X	0.8797	0.8595	0.8694	0.6986	0.8261	0.7570
YOLOv12L	0.8892	0.8631	0.8759	0.7692	0.7827	0.7759
YOLOv12N	0.8671	0.8901	0.8784	0.7569	0.7406	0.7487

Figure 2: Evaluation Metrics

, demonstrates partially conflicting results.

It is important to note that Sapkota et al. (2025) utilised the newer YOLOv12 architecture and the Base variant of RF-DETR (with 29M parameters, slightly fewer than the Nano variant used in the Robinson study).

- **Single-Class Detection:** In the context of single-class object detection, the YOLOv12N model outperformed the RF-DETR architecture.
- **Multi-Class Detection:** Conversely, the ranking shifts for multi-class detection, where the RF-DETR model demonstrated superior performance compared to YOLOv12N.

This discrepancy highlights the critical need to define the intended application: the optimal choice depends heavily on whether the scenario involves single-class or multi-class detection.

Research Gap and Contribution

The contradictory performance findings observed in the literature, particularly the trade-off between model size and average precision Robinson et al. (2025) and the impact of class complexity Sapkota et al. (2025), underscore the need for further validation.

The present paper connects with this existing body of literature by providing an empirical, comparative evaluation of the YOLO and RF-DETR architectures under the specific constraints of glass defect detection and data imbalance. This aims to serve as a specific supplementary reference for the performance of these models in a highly sensitive industrial application.

2. Methodology

2.1. Datasets and Assumptions

The search for suitable data sets proved to be challenging due to the inherent imbalance in industrial production. As mentioned in the introduction, the lack of data is a key problem in the context of defect detection. For the experimental implementation, two separate data sets were selected. These data sets represent different product forms, but have the identical defect type of gas bubbles.

Dataset A (Glasjars, Roboflow)

The first dataset was an annotated dataset of glass jars. This dataset was obtained from the Roboflow platform.

- **Defined classes:** This dataset contained two primary classes: the defect class (the gas bubble itself) and the container class (the glass container as an object). The models were trained to locate both the surrounding product and the specific defect using bounding boxes.
- **Scope and division:** The dataset comprised a total of 1728 images. The division into the three essential subcategories of training set, validation set ('val') and test set ('test') was already predefined. This division was adopted for the subsequent analysis.



Figure 3: Raw example image of the dataset A

Figure 3 shows that the images were captured in monotone coloring, they are not auto-rotated, meaning the angle of the object is different in every picture.

Dataset B (Glass Plates, Kaggle)

The second dataset was obtained from the Kaggle platform and was used to verify the model's performance on structurally different image data.

- **Scope and distribution:** With a total of 208 images, this dataset was significantly smaller than dataset A. Here, too, the distribution into training, validation and test sets was predefined and was adopted for the analysis.
- **Product and class definition:** In contrast to the first data set, data set B did not contain images of entire pharmaceutical containers, but rather images of a glass plate that exhibited the defect. The classification was limited exclusively to the defect class (gas bubble). This set came annotated with the respective bounding boxes and the class "DEFECT".
- **Important Feature:** A key feature of this data set was the guarantee that each image in the sample contained at least one defect. This represented a contrasting data basis to real production environments, as the natural imbalance between defective and intact samples was not represented here.

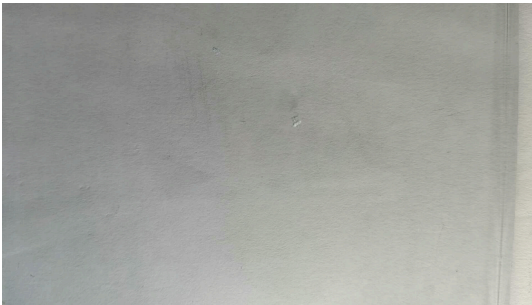


Figure 4: Example image of dataset B

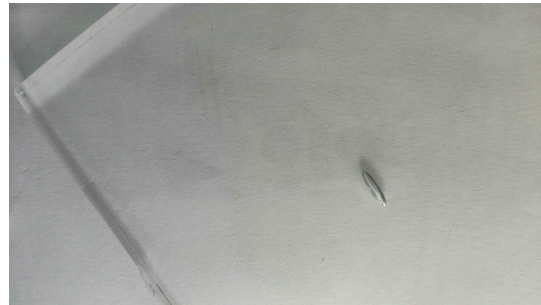


Figure 5: Example image of dataset B

As shown in (Figure 4) and Figure (Figure 5), the extent of the gas bubbles varied significantly from microlesions to large-area defects. This variation placed demands on the localisation capability of the models across a wide range of sizes.

In view of the structural and qualitative differences between the data sets, a number of assumptions were formulated in advance.

It was assumed that data set A would be more challenging due to the different rotation angles of the images and the more complex classification. A longer training period and lower overall performance were expected here compared to data set B.

On the other hand, the small size of dataset B led to the hypothesis of low generalisability, although rapid learning was assumed on the internal test images.

2.2. Model Selection

For the comparative part of the study, the decision was focused on two of the currently best-known architectures for object detection. This selection aimed to evaluate the performance of a conventional convolutional neural network (CNN)-based approach against a transformer-based model.

Ansprechen was in den ground truth files drin steht mit superclass, für was coco steht und warum es anders ist. dass die daten von kaggle auf roboflow von yolo zu coco konvertiert wurden

- **YOLO (You Only Look Once)** The YOLO model was selected as the first reference detector. The model was developed by the software developer Ultralytics and is considered a real-time object detection and segmentation model. It is characterised by its high speed and efficient performance, making it one of the most commonly used one-stage detectors.(Ultralytics (2025)) The YOLO Model's folder structure has the following components that are the same for every dataset. It consists of the aforementioned three folders: 'train', 'val', 'test' and in addition, a YAML source file (Yet Another Markup Language) was used. This specified the file paths to the respective folders and defined the number and names of the classes contained in the data set. Each of the main folders are subdivided into two subfolders:
 - One containing the image data without annotations.
 - The other containing five numbers: The class id, center_x, center_y, width and height of the bounding box
- **RF-DETR (Roboflow Detection Transformer)** The folder structure required for the RF-DETR model differed fundamentally from the YOLO standard, necessitating the use of the Common Objects in Context (COCO) format.

Similar to the preceding model, the directory maintained the main divisions (train, val, test). However, the key architectural distinction resided in the annotation storage. Instead of

utilizing individual label files per image, each main folder contained a single JSON file that held all annotation information for the respective subset.

The structure of this JSON file adhered to the COCO standard, encompassing the following five key elements:

1. Dataset Information: General metadata regarding the dataset itself.
2. License Information: Details concerning data usage rights.
3. Categories: A definition of the classes present in the dataset. A specific feature of the COCO format was the mandatory inclusion of a Supercategory (e.g., 'Animals'), under which individual classes (e.g., 'Cat') were assigned.
4. Image Information: Metadata for each image, such as file name and dimensions.
5. Annotation Information: Detailed annotation data for each instance, including the object's id, the corresponding image_id, the category_id, the bounding box coordinates, the area (in square pixels), and the segmentation information (segmentation and is_crowd variables)

For the experimental evaluation, the Nano variant (YOLOv11n, RFDETR-nano) was deliberately selected for both the YOLO and the RF-DETR architecture.

2.3. Model Mechanics

YOLO (You Only Look Once)

The YOLO (You Only Look Once) model is a real-time object detection system which fundamentally differs from traditional multi-stage detectors by processing the input image in a single pass. This architectural choice classifies YOLO as a Single-Shot Detector (SSD). This type of detector employs an algorithm that predicts the bounding box coordinates and the corresponding class probability of the detected object simultaneously and in one step. The primary beneficial consequence of this unified approach is a significant increase in processing speed. (Buhl (2024)) Consequently, the simultaneous prediction of the object's presence and its precise location accelerates the overall detection process, making it highly suitable for real-time applications.

RF-DETR (Roboflow Detection Transformer)

As the comparative alternative architecture, the RF-DETR (Roboflow Detection Transformer) model was selected. This model represents a real-time, state-of-the-art transformer-based architecture specifically designed for object detection and instance segmentation. (Robinson et al. 2025).

The RF-DETR architecture leverages a Vision Transformer (ViT) to effectively extract multiscale features from the input image.

Following feature extraction, the model utilizes a balanced combination of two distinct attention mechanisms:

Windowed Attention Blocks: These restrict the self-attention mechanism to a local neighbourhood, which significantly improves processing speed.

Non-Windowed Attention Blocks: These enable global interaction across features, which contributes to overall detection accuracy.

This deliberate balance helps maintain high performance while ensuring the model remains fast enough for real-time applications. Subsequently, the deformable cross-attention layers and the segmentation head bilinearly interpolate the output. Finally, the detection and segmentation losses are applied to all decoder layers during the training process (Robinson et al. 2025).

YOLO vs RF-DETR

The selection of YOLO and RF-DETR for the experimental evaluation is based on their fundamental architectural divergence. Although both models are utilized for the same purpose, object detection and localization of defects, they differ significantly in their internal working structure.

2.4. Model Training

The training procedure for both the YOLO and RF-DETR architectures was carried out under identical computational and hyperparameter conditions to ensure a fair and direct comparative evaluation.

- **Computational Environment:** All training and validation processes were conducted on the Google Colab Pro environment, utilizing an NVIDIA T4 GPU equipped with 16GB of VRAM (Video Random Access Memory).

- **Hyperparameters:** Both models were trained for a fixed duration of 100 epochs. A total batch size of 16 and a learning rate of 0.001 were applied consistently across all four training runs (YOLO on Datasets A and B; RF-DETR on Datasets A and B). The other Hyperparameters were left as they are configured when starting the training.

2.5. Model Testing

The final testing of the trained models was conducted under the same computational conditions as the training phase. For evaluation, the separate test splits of both Datasets A and B were used exclusively. This ensured that the model performance metrics were derived from data the models had not previously encountered during the training or validation phases.

2.6. Evaluation Metrics

The performance of the models was assessed using standard metrics commonly applied in object detection and machine learning. These metrics were calculated based on the following fundamental classifications:

- **True Positives (TP):** The model correctly identified a defect, and the predicted bounding box sufficiently overlapped with the ground truth annotation.
- **True Negatives (TN):** The model correctly identified the absence of a defect in an area where none was present.
- **False Positives (FP):** The model predicted a defect where the ground truth showed none (a false alarm).
- **False Negatives (FN):** The model failed to predict a defect where one was present (a missed detection).

Given these fundamental metrics, the following performance indicators were calculated:

- **Precision:** This metric measures the accuracy of the positive predictions, indicating the proportion of positively predicted instances that were truly correct.

$$Precision = \frac{TP}{TP + FP}$$

- **Recall:** This metric measures the model's ability to find all actual positive instances, indicating the proportion of actual defects that were successfully located.

$$Recall = \frac{TP}{TP + FN}$$

- **F1-Score:** The F1-Score represents the harmonic mean of Precision and Recall, providing a single metric that balances both factors.

$$F1 - Score = \frac{Precision \cdot Recall}{Precision + Recall}$$

- **Intersection over Union (IoU):** IoU measures the degree of overlap between the model's predicted bounding box and the ground truth bounding box. It is calculated as the area of intersection divided by the area of union of the two boxes.

$$IoU = \frac{TP}{(TP + FP + FN)}$$

- **mAP@50 (Mean Average Precision at IoU 50):** The mAP is the mean of the Average Precision (AP) across all classes. The AP, in turn, represents the Area Under the Precision-Recall Curve. The suffix @50 indicates that only detections with an IoU overlap of at least 50% with the ground truth bounding box were considered true positives for the calculation.

$$mAP@50 = \frac{1}{N} \sum_{i=1}^N AP_i@50$$

3. Results

3.1. Model Results and Statistical Evaluation

Yolo Kaggle, traintime = ~7min Yolo Roboflow, traintime = RF-DETR Kaggle RF-DETR

Roboflow, traintime ~ 6h ## Evaluation Visualization

4. Discussion

4.1. Interpretation of Results

(Self Note;)Buhl (2024) sagt dass das modell weniger false positives hat, unbedingt mit dem resultat von rf detr abgleichen!

4.2. Research Questions & Assumptions

Assumptions Es wurde angenommen, dass der Datensatz A eine längere Durchlaufzeit benötigt, da er grösser ist. Dies wurde im Abschnitt Model Results and Statistical Evaluation dargelegt.

5. Conclusion

5.1. Summary of Key Findings

5.2. Limitations and Challenges

The execution of the comparative study was subject to several inherent constraints and practical challenges:

- **Computational Constraints and Training Duration:** A significant limitation was the dependency on high-performance computing resources. It was observed that without a powerful GPU for fine-tuning, researchers must account for extended waiting periods. The training time increases proportionally with the size and complexity of the dataset, posing a barrier to rapid experimentation.
- **Discrepancy in Model Outputs:** Due to the divergent underlying model infrastructures (i.e., the unique software libraries and frameworks used by YOLO and RF-DETR), the final training results were represented differently. This resulted in variations in data storage formats, metrics calculated, and output visualizations. Consequently, considerable effort was required to unify the various outputs into a consistent and comparable format suitable for a fair performance comparison.
- **Data Scarcity and Suitability:** A major initial challenge, as previously discussed, was the difficulty in obtaining suitable public datasets containing sufficient annotated samples for the specific application of glass defect detection. The inherent data imbalance in the industrial context restricted the scope and depth of the training process.

5.3. Future Research Directions

The findings of this project open up several promising avenues for future research and applied development:

- **Real-Time Video Stream Evaluation:** A natural extension would be to test the trained models' performance using a real-time video signal. This would enable the precise evaluation of detection latency and practical precision under continuous operational conditions, moving closer to a live industrial application.

- **Edge Device Deployment Simulation:** It would be highly valuable to deploy the final trained weights onto a resource-constrained edge computing device (e.g., a Raspberry Pi), paired with a camera. This could simulate a real application scenario, allowing for the measurement of the actual processing speed (FPS) achievable on lightweight hardware, thus validating the choice of the Nano variants.
- **User Interface Development:** The creation of a user interface (UI) would enhance the usability of the models. This UI could visualize the results in a user-friendly manner, clearly distinguishing between defect and no defect conditions for production personnel.
- **Industrial Partnership and Validation:** Ideally, these proposed extensions could be executed in collaboration with an industrial partner. Such a partnership would facilitate the evaluation of whether these modern deep learning methods surpass the existing, often complex and expensive, traditional machine learning algorithms currently used for automated quality control in the pharmaceutical sector.

6. References

- Bhatt, Prahar M., Rishi K. Malhan, Pradeep Rajendran, Brual C. Shah, Shantanu Thakar, Yeo Jung Yoon, and Satyandra K. Gupta. 2021. "Image-Based Surface Defect Detection Using Deep Learning: A Review." *Journal of Computing and Information Science in Engineering* 21 (4): 040801. <https://doi.org/10.1115/1.4049535>.
- Buhl, Nikolaj. 2024. "YOLO Object Detection Explained: Evolution, Algorithm, and Applications." *Https://Encord.com/*. [https://encord.com/blog/yolo-object-detection-guide/#:~:text=YOLO%20\(You%20Only%20Look%20Once,identify%20objects%20in%20the%20image](https://encord.com/blog/yolo-object-detection-guide/#:~:text=YOLO%20(You%20Only%20Look%20Once,identify%20objects%20in%20the%20image).
- Foglia, Pierfrancesco, Cosimo Antonio Prete, and Michele Zanda. 2015. "An Inspection System for Pharmaceutical Glass Tubes." *WSEAS TRANSACTIONS on SYSTEMS Archive* 14: 123–36. <https://api.semanticscholar.org/CorpusID:62820567>.
- Robinson, Isaac, Peter Robicheaux, Matvei Popov, Deva Ramanan, and Neehar Peri. 2025. "RF-DETR: Neural Architecture Search for Real-Time Detection Transformers." <https://arxiv.org/abs/2511.09554>.
- Sapkota, Ranjan, Rahul Harsha Cheppally, Ajay Sharda, and Manoj Karkee. 2025. "RF-DETR Object Detection Vs YOLOv12 : A Study of Transformer-Based and CNN-Based Architectures for Single-Class and Multi-Class Greenfruit Detection in Complex Orchard Environments Under Label Ambiguity." <https://arxiv.org/abs/2504.13099>.
- Ultralytics. 2025. "Object Detection Datasets Overview." <https://docs.ultralytics.com/de/datasets/detect/#ultralytics-yolo-format>.

7. Lists

List of Figures

Figure 1	Comparison Table of different Object Detectors	7
Figure 2	Evaluation Metrics	8
Figure 3	Raw example image of the dataset A	10
Figure 4	Example image of dataset B	11
Figure 5	Example image of dataset B	11

List of Tables

8. Attachments

8.1. Raw Data

Dataset A

YOLO-Model *Training Testing* **RF-DETR** *Training Testing* ### Dataset B **YOLO-Model**
Training Testing **RF-DETR** *Training Testing*